Claude Support

Search for articles...

All Collections > Claude > Features and capabilities > How to create custom Skills

How to create custom Skills

Updated over 3 weeks ago

Table of contents

Skills are available for users on free, Pro, Max, Team, and Enterprise plans. This feature requires code execution to be enabled. Skills are also available in beta for Claude Code users and for all API users using the code execution tool.

Custom Skills let you enhance Claude with specialized knowledge and workflows specific to your organization or personal work style. This article explains how to create, structure, and test your own Skills.

Skills can be as simple as a few lines of instructions or as complex as multi-file packages with executable code. The best Skills:

- Solve a specific, repeatable task
- Have clear instructions that Claude can follow
- Include examples when helpful
- Define when they should be used
- Are focused on one workflow rather than trying to do everything

Create a Skill.md file

Every Skill consists of a directory containing at minimum a Skill.md file, which is the core of the Skill. This file must start with a YAML frontmatter to hold name and description fields, which are required metadata. It can also contain additional metadata, instructions for Claude or reference files, executable scripts, or tools.

Required metadata fields

name: A human-friendly name for your Skill (64 characters maximum)

- **Example:** Brand Guidelines

description: A clear description of what the Skill does and when to use it.

- This is critical—Claude uses this to determine when to invoke your Skill (200 characters maximum).
- **Example:** Apply Acme Corp brand guidelines to presentations and documents, including official colors, fonts, and logo usage.

Optional metadata fields

dependencies: Software packages required by your Skill.

- **Example:** python>=3.8, pandas>=1.5.0

The metadata in the Skill.md file serves as the first level of a progressive disclosure system, providing just enough information for Claude to know when the Skill should be used without having to load all of the content.

Markdown body

The Markdown body is the second level of detail after the metadata, so Claude will access this if needed after reading the metadata. Depending on your task, Claude can access the Skill.md file and use the Skill.

Example Skill.md

Brand guidelines skill

```
## Metadata
name: Brand Guidelines
description: Apply Acme Corp brand guidelines to all presentations and documents

## Overview
This Skill provides Acme Corp's official brand guidelines for creating consister

## Brand Colors
```

```
Our official brand colors are:
- Primary: #FF6B35 (Coral)
- Secondary: #004E89 (Navy Blue)
- Accent: #F7B801 (Gold)
- Neutral: #2E2E2E (Charcoal)

## Typography

Headers: Montserrat Bold
Body text: Open Sans Regular
Size guidelines:
- H1: 32pt
- H2: 24pt
- Body: 11pt

## Logo Usage

Always use the full-color logo on light backgrounds. Use the white logo on dark

## When to Apply

Apply these guidelines whenever creating:
- PowerPoint presentations
- Word documents for external sharing
- Marketing materials
- Reports for clients

## Resources

See the resources folder for logo files and font downloads.
```

Add resources

If you have too much information to add to a single Skill.md file (e.g., sections that only apply to specific scenarios), you can add more content by adding files within your Skill directory. For example, add a REFERENCE.md file containing supplemental and reference information to your Skill directory. Referencing it in Skill.md will help Claude decide if it needs to access that resource when executing the Skill.

Add scripts

For more advanced Skills, attach executable code files to Skill.md, allowing Claude to run code. For example, our document skills use the following programming languages and packages:

- Python (pandas, numpy, matplotlib)
- JavaScript/Node.js
- Packages to help with file editing
- visualization tools

Note: Claude and Claude Code can install packages from standard repositories (Python PyPI, JavaScript npm) when loading Skills. It's not possible to install additional packages at runtime with API Skills—all dependencies must be pre-installed in the container.

Package your skill

Once your Skill folder is complete:

1. Ensure the folder name matches your Skill's name.
2. Create a ZIP file of the folder.
3. The ZIP should contain the Skill folder as its root (not a subfolder).

Correct structure:

```
my-Skill.zip
  └── my-Skill/
    ├── Skill.md
    └── resources/
```

Incorrect structure:

```
my-Skill.zip
  └── (files directly in ZIP root)
```

Test your skill

Before uploading

1. Review your Skill.md for clarity
2. Check that the description accurately reflects when Claude should use the Skill
3. Verify all referenced files exist in the correct locations
4. Test with example prompts to ensure Claude invokes it appropriately

After uploading to Claude

1. Enable the Skill in **Customize > Skills**.
2. Try several different prompts that should trigger it
3. Review Claude's thinking to confirm it's loading the Skill
4. Iterate on the description if Claude isn't using it when expected

> **Note for Team and Enterprise plans:** To make a skill available to all users in your organization, see **Provision and manage Skills for your organization**.

Best practices

Keep it focused: Create separate Skills for different workflows. Multiple focused Skills compose better than one large Skill.

Write clear descriptions: Claude uses descriptions to decide when to invoke your Skill. Be specific about when it applies.

Start simple: Begin with basic instructions in Markdown before adding complex scripts. You can always expand on the Skill later.

Use examples: Include example inputs and outputs in your Skill.md file to help Claude understand what success looks like.

Test incrementally: Test after each significant change rather than building a complex Skill all at once.

Skills can build on each other: While Skills can't explicitly reference other Skills, Claude can use multiple Skills together automatically. This composability is one of the most powerful parts of the Skills feature.

Review the open Agent Skills specification: Follow the guidelines at **agentskills.io**, so skills you create can work across platforms that adopt the standard.

For a more in-depth guide to skill creation, refer to **Skill authoring best practices** in our Claude Docs.

Security considerations

- Exercise caution when adding scripts to your Skill.md file.
- Don't hardcode sensitive information (API keys, passwords).
- Review any Skills you download before enabling them.
- Use appropriate MCP connections for external service access.

Example skills to reference

Visit our repository on GitHub for example Skills you can use as templates: https://github.com/anthropics/skills/tree/main/skills.

Related Articles

- Create and edit files with Claude
- What are Skills?
- Use Skills in Claude
- Use Claude for PowerPoint
- Browse skills, connectors, and plugins in one directory

Did this answer your question?

Product

Research

Company

News

Careers

Terms of Service - Consumer

Terms of Service - Commercial

Privacy Policy

Usage Policy

Responsible Disclosure Policy

Compliance